RECONCILE

Technical Specification v5.0

A Controller-Reconciliation Runtime for Institutional Software

Credit IQ

March 2026

Core Thesis

AI commoditizes interfaces. Value lives in system architecture. This specification defines the runtime that turns architectural primitives into executable governance infrastructure.

Table of Contents

(Generate in Word: References > Table of Contents)

Part I: Philosophy & Foundations

1. Introduction

Reconcile is a runtime framework for building institutional software: banking, insurance, government, healthcare, audit infrastructure, and any domain where decisions, roles, policies, and state transitions must be coordinated across time.

Most enterprise systems are built as giant workflow engines that become rigid and unmaintainable. Reconcile inverts this using an insight proven by Kubernetes, React, Temporal, event sourcing, and CRDTs:

> **The Reconciliation Principle**
>
> Instead of one big orchestration brain, build a small declarative core and many tiny controllers that each reconcile one invariant. The system emerges from the composition of independent feedback loops, not from central coordination.

Target domains: Banking, insurance, government, healthcare, supply chain, legal compliance, audit, cooperative finance.

What Reconcile is NOT: Not a UI framework, chatbot wrapper, data pipeline, or general-purpose workflow engine. It is governance runtime infrastructure.

2. Design Principles

2.1 Declarative Over Imperative

Developers describe WHAT the system is (states, roles, policies, invariants, transitions), never HOW the runtime enforces it.

2.2 Deterministic Core, Advisory Agents

State transitions, policy evaluation, and permission checks are deterministic. AI agents observe events and propose actions, but the deterministic core validates and constrains them. A system must function correctly without AI; AI makes it faster, not possible.

2.3 Resource-Primary, Event-Native

Resources are operational truth (what you query, what regulators examine). Events are the mechanism of change (append-only log). Every resource state is derivable from replaying its event stream. When they disagree, events are authoritative.

2.4 Controller-Based Composition

Each controller runs one reconciliation loop: observe state, compare to invariant, act if different. Controllers are independently deployable, testable, and replaceable.

2.5 Invariant-Driven Design

Every controller exists to enforce an explicit invariant. Invariants are declared in the system spec, not buried in code. This makes the system formally verifiable.

2.6 Observable by Default

Audit trails are structural, not bolt-on. Every state change records who, what, when, in what role, at what authority level, under which policies, and with what outcome.

2.7 Plugin Extensibility

The kernel does seven things and nothing else. Everything beyond is a plugin.

3. Formal System Model

Reconcile is described as a 9-tuple:

Formal Model

$\mathcal{R} = (S, E, T, P, R, V, I, D, G)$

Symbol	Name	Definition
S	States	Finite set of lifecycle stages. $S = \{s_1, s_2, ..., s_n\}$
E	Events	Append-only sequence of immutable records. $E = [e_1, e_2, ...]$
T	Transitions	Relation $T \subseteq S \times S$ with guards $g: T \to \{true, false\}$
P	Policies	Constraint functions. $P = \{p: (Resource, Context) \to \{allow, deny\}\}$
R	Roles	Permission mappings. $R = \{r: (action, type, state) \to \{true, false\}\}$
V	Invariants	System properties. $V = V_s$ (strong) $\cup$ V_e (eventual). May be defined over subgraphs of G
I	Instruments	Controllers and agents that reconcile toward invariants
D	Desired State	Target state for each resource. D: resource_id → state?
G	Graph	$G = (N, \Xi)$ where N = resource instances (nodes), Ξ = typed relationships (edges)

3.1 Transition Function

A transition $\tau(r, s', actor, role)$ is valid iff:

1. $(r.state, s') \in T$ (structural validity)
2. $R(role, transition, r.type, s') = true$ (authorization)
3. $g(r.state, s')(r) = true$ for all guards (preconditions)
4. $P(r, ctx) = allow$ for all applicable policies (constraints)
5. $\forall v \in V: v(SystemState') = true$ (invariant preservation)

3.2 Convergence Guarantee

The event cascade (event → controller → event → controller) must converge. Guaranteed by:

- **Resource-level convergence:** Controllers must move resources toward desired states or terminal states. Each cascade step reduces the distance between current_state and desired_state. This is the fundamental guarantee — stronger than static DAG analysis because it handles dynamic dependencies.
- **Monotonicity:** The system defines a partial order on states. Controllers can only produce transitions that move 'forward' in this order within a single cascade. Backward transitions require a new external event (human action, timeout).
- **Depth bounding:** Max cascade depth (default: 10). Exceeding emits cascade.depth_exceeded event. This is a safety net, not the primary guarantee.

Static DAG analysis at registration time catches obvious cycles. But the real convergence guarantee is resource-level: controllers reconcile toward desired state, and the distance to desired state strictly decreases.

3.3 Event Ordering Axiom

> **Ordering Guarantee**
>
> Events for a given resource_id are totally ordered. Partition key = resource_id. This guarantees deterministic replay.

Cross-resource events are only partially ordered. Cross-resource coordination uses sagas or constraint controllers (Section 27).

Part II: The Eight Primitives

Primitive	What It Is	Example
States	Lifecycle stages	APPLIED → UNDERWRITING → APPROVED
Roles	Who can do what, where	Underwriter can approve in UNDERWRITING
Policies	Conditional decision rules	Loans > ₹50L need committee approval
Events	Immutable record of everything	loan.transitioned {from: APPLIED, to: UW}
Invariants	Properties that must always hold	Total exposure per applicant < ₹10Cr
Desired State	Target state for reconciliation	desired: APPROVED, current: UNDERWRITING
Observability	Structural audit trails	Who, what, when, which policies, outcome
Time	Temporal context for decisions	Policy version at decision point

4. States

```
State(name: str, status: ACTIVE | TERMINAL | SUSPENDED, metadata: dict)

StateMachine(states: [State], transitions: [Transition], initial_state: str)

Transition(from_state, to_state, guard: Callable?, required_role: str?)
```

Guards are pure functions encoding preconditions. Terminal states have no outbound transitions. The runtime rejects any transition not explicitly defined.

4.1 First-Class Resource Objects (DSL)

```
loan = resource("Loan", schema={"amount": "number", "applicant_id": "string"})

loan.lifecycle(initial="APPLIED",
transitions=[("APPLIED","UNDERWRITING"), ...])
```

5. Roles

```
Role(name: str, permissions: [Permission])

Permission(action: str, resource_type: str, in_states: tuple[str])
```

5.1 Permission Shorthand

```
"view" → Permission(action="view", in_states=("*",))

"transition:UNDERWRITING" → Permission(action="transition",
in_states=("UNDERWRITING",))
```

6. Policies

```
Policy(name, description, evaluate: Callable, applicable_states, priority)

PolicyResult(passed: bool, message: str, details: dict)
```

6.1 Policy Indexing

```
PolicyIndex: (resource_type, current_state) → [applicable Policy]
```

Only matching policies are evaluated per transition. Prevents O(n) policy scans.

6.2 Policy Types

Type	Description	Example
Constraint	Blocks transitions	Loans > 50L require committee
Requirement	Precondition	All docs before underwriting
Approval	Role sign-off	Manager before disbursement
Escalation	Auto-escalate	Risk > 0.8 → risk committee
Temporal	Time-based	Approval expires after 30 days
Cross-resource	Across entities	Applicant exposure < 10Cr
Regulatory	Compliance	RBI circular limits

7. Events

```
Event(id, type, resource_id, data, timestamp, actor, authority_level)
```

7.1 System Events

Pattern	Trigger	Data
{resource}.created	Creation	state, data
{resource}.transitioned	Transition	from, to, actor, role, authority_level
transition.rejected	Invalid transition	from, to, reason
policy.violated	Policy failure	policy, message
agent.proposal	Agent proposal	agent, action, confidence
invariant.violated	Invariant failure	invariant, violation

7.2 Event Log

- Append-only, offset-based, per-resource total ordering
- Pattern subscriptions (exact, prefix, wildcard)
- Consumer offset tracking for at-least-once processing
- Backends: Postgres append-only table, Kafka, Redis Streams

7.3 Event Log Snapshots

Append-only logs grow forever. Without mitigation, replay from offset zero becomes prohibitively expensive. The system supports snapshots:

```
Snapshot(resource_id, state, data, version, event_offset, timestamp)
```

A snapshot captures resource state at a given event offset. Replay can start from the most recent snapshot instead of event zero. This is the same pattern banks use (balance-forward records) and Kafka uses (log compaction with snapshots).

- Snapshots are created periodically or on significant state changes
- Replay: load latest snapshot, then replay events from snapshot.event_offset
- Old events before the oldest active snapshot may be archived or compacted
- Snapshots are themselves immutable records stored alongside the event log

8. Invariants

> **Why Invariants**
>
> Making invariants explicit elevates implicit controller behavior into verifiable system properties. You can enumerate every guarantee, test independently, and prove correctness.

```
Invariant(name, description, check: Callable, scope: resource|transition|
cross_resource|system)

InvariantResult(holds: bool, violation: str?, details: dict)
```

8.1 Strong vs Eventual Invariants

Not all invariants can be checked synchronously inside a transition. The system distinguishes two enforcement modes:

Type	Enforcement	Example	Checked When
Strong	Synchronous, inside transaction	amount > 0	Before state change commits
Eventual	Asynchronous, by controllers	exposure < limit	After commit, by constraint controller

Strong invariants block transitions immediately. Eventual invariants are enforced by dedicated controllers that run after the event cascade. If an eventual invariant is violated, the controller emits an invariant.violated event and may trigger compensating actions (reversal, suspension, escalation).

This mirrors how databases work: CHECK constraints are strong (fail immediately), while triggers and materialized views are eventual (enforce asynchronously).

8.2 Invariant Scopes

Scope	Checks	Typical Mode	Example
Resource	Single resource	Strong	Amount > 0
Transition	Before/after state	Strong	State moves toward terminal
Cross-resource	Across resources	Eventual	Applicant exposure < limit
System-wide	Global property	Eventual	No conflicting states

8.3 Controllers as Invariant Implementations

```
controller("exposure-checker", enforces=["exposure_limit"], ...)
```

Traceable chain: Invariant → Controller → Events. For eventual invariants, the controller IS the enforcement mechanism.

8b. Desired State

> **The Missing Piece**
>
> Kubernetes reconciles current state toward desired state. Most workflow engines reconcile against events. Adding Desired State completes the Kubernetes analogy and gives controllers a clear target: push the resource from where it is to where it should be.

```
DesiredState(resource_id: str, target_state: str, requested_by: str,
authority: str)
```

When a desired state is set (by a human, controller, or agent proposal), the reconciliation loop activates:

```
while resource.state != resource.desired_state:
    for controller in controllers:
        controller.reconcile(resource)  // push toward desired
```

This creates three modes of operation:

Mode	Trigger	Example
Imperative	Direct transition request	system.transition(loan, "APPROVED")
Declarative	Set desired state, let controllers reconcile	resource.desired_state = "DISBURSED"
Reactive	Event triggers controller, which sets desired state	on loan.scored → set desired APPROVED

Imperative transitions are syntactic sugar for: set desired state + run reconciliation. The underlying mechanism is always desired state reconciliation.

9. Observability & Audit

```
AuditRecord(resource_type, resource_id, actor, role, authority_level,
            previous_state, new_state, policies_evaluated/passed/failed,
            invariants_checked, timestamp)
```

Audit log records ONLY state transitions with full decision context. Event log records everything.

10. Temporal Complexity

```
TemporalContext(effective_date, policy_version, fiscal_year)
```

Attached to every transition. Answers: "What rules applied when this decision was made?"

Part III: Kernel Architecture

Eight subsystems. Target: 5,000-7,000 LOC.

Subsystem	Responsibility	Analogous To
Resource Registry	Type definitions, schema, desired state	K8s CRDs + spec
State Store	Current + desired resource state	etcd
Event Log + Snapshots	Append-only history with snapshots	Kafka + compaction
Policy Engine	Constraint evaluation (with indexing)	K8s admission controllers
Invariant Checker	Strong (sync) + eventual (async) invariants	DB CHECK + triggers
Controller Scheduler	Event dispatch + reconciliation loop	K8s controller manager
Audit Log	Regulatory compliance	Court record
Snapshot Store	Point-in-time resource state for replay	Bank balance-forward

11. Transition Authority Levels

Level	Source	Audit Implications	Restrictions
HUMAN	User via API/UI	Full actor+role attribution	Subject to all RBAC
CONTROLLER	Reactive/reconciliation ctrl	Attributed to controller name	Subject to policies; no RBAC override
AGENT	AI proposal accepted	Agent name + confidence	Must pass policies; may need human confirm
SYSTEM	Internal kernel op (timeout)	Attributed to "system"	Bypasses RBAC; subject to policies+invariants

Every audit record and event includes authority_level. Eliminates audit trail ambiguity.

12. Transaction Boundary

```
BEGIN
  1. validate_state_machine()
  2. check_role_permissions()
  3. evaluate_guards()
  4. evaluate_policies()      // via policy index
  5. verify_invariants()       // explicit check
  6. apply_state_change()
```

```
  7. write_audit_record()      // includes authority_level
  8. emit_events()
COMMIT
```

Steps 1-8 are atomic. If any fails, entire transition rolls back.

13. Event Cascade Termination

After commit, events flow: event → controller → event → controller. Termination guaranteed by:

- **Resource-level convergence:** Controllers push resources toward desired_state. Distance to desired_state strictly decreases each step.
- **Monotonicity:** Partial order on states. Controllers move forward only within a cascade.
- **Depth bound:** Default max 10. Safety net, not primary guarantee.

Static DAG analysis at registration catches obvious cycles. Dynamic cycles are caught by the convergence check: if a resource's state is not closer to desired_state after a controller runs, the cascade halts for that resource.

14. SystemContext (Safe API)

```
SystemContext.transition(resource_id, to_state, actor, role, authority_level)
SystemContext.emit(event_type, resource_id, data)
SystemContext.get_resource(resource_id)
SystemContext.check_invariant(invariant_name)
SystemContext.query_audit(resource_id) / query_events(...)
```

15. Concurrency Model

	v1 (MVP)	v2 (Scale)	v3 (Production)
Events	Single processor	Partitioned by resource_id	Worker pools
Controllers	Sequential	Parallel (independent)	Distributed
State Store	In-memory	PostgreSQL	Sharded
Event Log	In-memory	Postgres append-only	Kafka

Part IV: Developer-Facing DSL

16. define_system()

```
system = define_system(name="loan",
    states=["APPLIED","UNDERWRITING","APPROVED","DISBURSED","REJECTED"],
    transitions=[("APPLIED","UNDERWRITING"),("UNDERWRITING","APPROVED"),...],
    roles={"officer":["view","transition:UNDERWRITING"],"manager":
["transition:*"]},
    policies=[...], invariants=[...], agents=[...],
reactive_controllers=[...])
```

17. Resource API

```
loan = await system.create({"applicant":"Acme","amount":5_000_000})
outcome = await system.transition(loan.id, "UNDERWRITING", actor="u7",
role="officer")
events = await system.events(resource_id=loan.id)
```

18. Controller DSL

```
class FraudCtrl(Controller):
    name="fraud"; priority=80; enforces=["no_fraud"]
    async def reconcile(self, resource, ctx): ...


auto_uw = define_controller(name="auto-uw", on_event=["loan.created"],
    reconcile=handler, authority_level="CONTROLLER")
```

19. Agent DSL

```
risk_agent = define_agent(name="risk", observe=["loan.created"],
propose=score_fn)
# Agents emit Proposals, never state changes
Proposal(agent, action, resource_id, data, confidence)
```

20. Invariant DSL

```
invariant("exposure_limit", scope="cross_resource", mode="eventual",
    check=lambda r,ctx: InvariantResult(holds=exposure(r)<limit))
invariant("positive_amount", mode="strong").on_resource("loan").ensure(lambda
r: r.data["amount"]>0)
```

20b. Desired State API

```
# Imperative (direct transition)
await system.transition(loan.id, "APPROVED", actor="mgr", role="manager")

# Declarative (set desired, let controllers reconcile)
await system.set_desired(loan.id, "DISBURSED", requested_by="mgr")
# Controllers will push: APPROVED → DISBURSED through valid transitions

# Query desired state
resource = await system.get(loan.id)
print(resource.state, resource.desired_state)  # APPROVED, DISBURSED
```

21. Plugin DSL

```
class NotifPlugin(Plugin):
    name="notifications"
    def install(self, kernel): kernel.register_reactive_controller(...)
kernel.use(NotifPlugin())
```

22. Architecture Statement

> **Template**
>
> "This system coordinates [X states] across [Y roles] under [Z policies] maintaining [W invariants] reconciling toward [desired states] over [T time horizon], with AI at [N decision nodes]."

Part V: Concurrency, Failure & Mature Requirements

23. Cross-Resource Transactions

Two loans approved simultaneously may jointly violate exposure limits. Strategies:

Strategy	How	Trade-offs
Serializable Txn	DB-level serialization	Strong; limits throughput
Sagas	Compensating transactions	Scalable; complex error handling
Constraint Controllers	Dedicated controller serializes scope	Simple; single contention point
Optimistic Validation	Post-commit check + compensate	Max throughput; needs compensation

Default: Constraint Controllers (v1). Sagas (v2+).

24. Failure Handling

- Controllers MUST be idempotent (at-least-once delivery)
- v1: at-least-once, total order. v2+: exactly-once via dedup, per-resource order
- Dead letter queue for unprocessable events
- Circuit breakers: repeated failures stop dispatch to failing controller

25. PostgreSQL Schema

```
CREATE TABLE resources (id UUID PK, type TEXT, state TEXT, desired_state TEXT,
    data JSONB, version INT, created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ);
CREATE TABLE events (id UUID PK, offset BIGSERIAL, type TEXT, resource_id
UUID,
    payload JSONB, actor TEXT, authority_level TEXT, timestamp TIMESTAMPTZ);
CREATE TABLE audit_log (id UUID PK, resource_type TEXT, resource_id UUID,
    actor TEXT, role TEXT, authority_level TEXT, previous_state TEXT,
    new_state TEXT, policies_evaluated TEXT[], invariants_checked
TEXT[], ...);
CREATE TABLE snapshots (id UUID PK, resource_id UUID, state TEXT,
    data JSONB, version INT, event_offset BIGINT, timestamp TIMESTAMPTZ);
CREATE INDEX idx_events_resource ON events (resource_id, offset);
CREATE INDEX idx_snapshots_resource ON snapshots (resource_id, event_offset
DESC);
```

26. Workflow Graphs (Future)

```
workflow("loan", sequence("APPLIED","UW"), parallel("DOC","RISK"),
```

```
    join("COMPLETE"), branch(on="type", paths={...}))
```

27. Agent Coordination

```
decision_node("committee", agents=["risk","fraud","market"],
aggregation="majority")
```

- confidence > 0.9: auto-accept. 0.5-0.9: human review. < 0.5: reject + escalate

28. Security

- Kernel enforces authorization only (authn from application layer)
- RBAC primary; ABAC for fine-grained. All decisions audited
- Encryption at rest; field-level PII encryption; tamper-evident audit (hash chains)

29. Multi-Tenancy

- Tenant isolation at resource/event/audit level
- Per-tenant policies, storage backends, and scoped contexts

30. API Layer

```
POST /api/{type}                  → Create resource
POST /api/{type}/{id}/transition  → Request transition
GET  /api/{type}/{id}/events      → Event history
GET  /api/{type}/{id}/audit       → Audit trail
WS   /api/events/stream?pattern=  → Real-time stream
```

31. Testing Infrastructure

```
harness = SystemTestHarness(system)
harness.assert_transition_succeeds("L-1", "UNDERWRITING")
harness.assert_transition_blocked("L-1", "APPROVED", policy="high_value")
harness.assert_invariant_holds("exposure_limit")
harness.replay_under_policies("L-1", policy_version="2026-Q2")
```

Part VII: Graph Foundations

Every cross-resource operation in the preceding sections is a graph operation. Resources reference other resources. Invariants span neighborhoods. Event history forms causal chains. The state machine is a directed graph. Making the graph explicit is not adding a feature — it is acknowledging what the architecture already computes.

> **Architectural Principle**
>
> The event log and state store remain the source of truth. Graphs are derived projections — materialized views built incrementally from events and state. The kernel never depends on graph traversal for its core transition logic. This keeps the kernel simple and deterministic while unlocking graph analytics, invariant checking, and GNN inference.

This dual-model architecture combines a deterministic state machine runtime with graph analytics and AI inference — a combination that is extremely rare. Most systems pick one. Reconcile supports all three by keeping them in the correct dependency order:

```
Event Log (ground truth)
    ↓
State Store (operational truth)
    ↓
Graph Projections (analytical truth)
    ↓
Controllers / Agents / Invariant Checkers (consumers)
```

The graph reads from the state store and event log. It never writes back to them. Data flows one direction.

37. The Three Graph Layers

Layer	What It Models	Source of Truth	Graph Is	Mutability
Schema	Type topology	System definition	Primary (static)	Changes only with system def
Instance	Live relationships	State store	Derived projection	Rebuilt incrementally from events
Temporal	Causal event history	Event log	Derived projection	Appended as cascades execute

Only the schema graph is graph-native (it is static and defined at system creation). The instance and temporal graphs are projections derived from the primary stores.

38. Schema Graph (Layer 1) — Primary

The schema graph defines what types exist, how they relate, and what lifecycles they follow. This layer IS graph-native because it is static — it changes only when the system definition changes, not during runtime operation.

38.1 Relationship Declaration

```
Edge(from_type, to_type, relation, cardinality: 1:1 | 1:N | N:1 | N:M,
required: bool)
```

38.2 DSL

```
loan = resource("Loan", schema={"amount": "number", "applicant_id": "string"})
applicant = resource("Applicant", schema={"name": "string", "pan": "string"})


loan.belongs_to(applicant)           # N:1, required
loan.secured_by(collateral)           # N:1, optional
loan.funds(disbursement)              # 1:N
```

38.3 Schema Graph Algorithms

Algorithm	Question	Example
Reachability	Can state X reach state Y?	Can REJECTED ever reach DISBURSED?
Shortest path	Minimum transitions?	Fewest steps from APPLIED to DISBURSED
Cycle detection	State machine loops?	Can a loan loop REVIEW ↔ UNDERWRITING?
Deadlock detection	Stuck non-terminal states?	State with no outbound and not terminal
Relationship closure	Transitive type reachability?	What does Loan indirectly touch?

These algorithms run at system definition time (compile-time validation), not at runtime. This opens the door to compile-time policy and invariant verification — the schema graph becomes a type system for institutional processes.

39. Instance Graph (Layer 2) — Derived Projection

> **Why Derived, Not Primary**
>
> Graph databases struggle with high-frequency transactional updates and deterministic state machines under concurrency. The core transition model — (resource_state, event, context) → new_state — is fundamentally a keyed state operation, not a graph traversal. Kubernetes, Kafka Streams, Temporal, and Git all use keyed state stores, not graph databases, for this reason.

39.1 Projection Architecture

```
State Store (Postgres / in-memory)    # source of truth for resource state
```

```
    ↓ graph builder (incremental)
Instance Graph (projection)           # materialized view for graph queries
```

The graph builder listens to events and incrementally updates the projection. When a loan is created with applicant_id=ACME-01, the builder creates a node for the loan and an edge to the applicant node. When a loan transitions, the builder updates the node's state property.

39.2 Nodes and Edges

```
Node = {id, type, state, desired_state, data, version}    # mirrors Resource
Edge = {from_id, to_id, relation, metadata}               # derived from
references
```

39.3 Graph Query Interface

Controllers and invariant checkers query the projection (read-only):

```
graph.neighbors(resource_id, edge_type?) → [Node]          # adjacent nodes
graph.subgraph(resource_id, depth=1) → Graph              # local neighborhood
graph.path(from_id, to_id) → [Edge]                       # relationship path
graph.aggregate(resource_id, edge_type, field, fn) → value # neighborhood
aggregation
graph.has_cycle(subgraph) → bool                          # cycle detection
graph.degree(resource_id, edge_type, filter?) → int       # connection count
```

Writes go through the normal kernel transition API, which updates the state store. The graph builder then updates the projection. Controllers never write to the graph directly.

39.4 Graph-Expressible Invariants

Invariants can be expressed as graph predicates over the projection:

Invariant	Graph Expression	Type
Exposure limit	aggregate(applicant, belongs_to, amount, SUM) < 10Cr	Neighborhood aggregation
No circular guarantees	has_cycle(guarantee_subgraph) == false	Cycle detection
Collateral coverage	aggregate(loan, secured_by, value, SUM) >= loan.amount	Neighborhood aggregation
Single active per collateral	degree(collateral, secured_by, state=ACTIVE) <= 1	Degree constraint
Approval chain complete	all_on_path(loan, DISBURSED) have audit_records	Path property

Strong invariants use single-node checks (no graph needed). Eventual invariants use graph queries over the projection. This separation keeps the transaction boundary fast while enabling complex cross-resource checks asynchronously.

39.5 Projection Backends

Backend	Use Case	Trade-offs
In-memory (NetworkX/rustworkx)	Testing, prototyping	Fast; no persistence; < 100K nodes
PostgreSQL + Apache AGE	Production	Single infra dependency; openCypher; good enough for most
Neo4j / Memgraph	High graph complexity	Native graph; Cypher; best for deep traversals

Recommended: in-memory for v1, PostgreSQL + Apache AGE for v2 (keeps Postgres as single infrastructure dependency), Neo4j for v3+ when graph query complexity demands native performance.

40. Temporal Graph (Layer 3) — Derived Projection

Built incrementally from the event log. When event A causes controller B to emit event C, the temporal graph builder adds a causal edge from A to C attributed to B.

```
CausalEdge(cause_event_id, effect_event_id, via: str)  # via = controller or
agent name
```

40.1 Causal Audit Traversal

- "Why was this disbursed?" → backward traversal from disbursement event
- "What resulted from this approval?" → forward traversal from approval event
- "Full decision chain?" → subgraph of all events + causal closure
- "Did any agent contribute?" → filter for agent.proposal nodes in causal subgraph

40.2 Event Log vs Temporal Graph

The event log is flat and ordered — it answers "what happened." The temporal graph adds causal structure — it answers "why." The event log is the source; the temporal graph is the projection.

41. Kernel Integration

41.1 Transaction Boundary (Unchanged from Part III)

The core transaction boundary does NOT include graph operations. The 9-step transaction writes to the state store and event log only. Graph projections are updated post-commit by the graph builder, which subscribes to the event log.

```
TRANSACTION (synchronous, in kernel):
  1-5. validate, check roles, guards, policies, strong invariants
  6-9. apply state change, write audit, emit events


POST-COMMIT (asynchronous, outside kernel):
```

```
graph builder updates instance graph projection
graph builder updates temporal graph projection
eventual invariant controllers query graph projections
```

This keeps the kernel at ~5-7K LOC. The graph builder and projection engine are separate subsystems — plugins, not kernel components.

41.2 SystemContext (Extended for Read-Only Graph Access)

```
SystemContext.graph.neighbors(resource_id, edge_type?) → [Resource]
SystemContext.graph.subgraph(resource_id, depth) → Graph
SystemContext.graph.aggregate(resource_id, edge_type, field, fn) → value
SystemContext.graph.causal_chain(event_id, direction) → [Event]
```

Graph access is namespaced under SystemContext.graph to make the read-only, derived nature explicit. Controllers query the graph; they never mutate it directly.

42. GNN Integration

> **MaayaAI Connection**
>
> The same GNN techniques used for attack path analysis in cybersecurity apply to institutional governance: embed the instance graph projection, learn normal patterns, flag anomalous subgraphs. The graph projection is the perfect substrate for GNN inference.

42.1 GNN as Agent

```
gnn_agent = define_agent(name="graph-anomaly",
    observe=["loan.created", "loan.transitioned"],
    propose=gnn_scoring_fn)
```

The GNN agent: (1) extracts features from the instance graph projection, (2) runs message-passing layers, (3) scores subgraphs for anomaly, (4) emits proposals. The graph projection provides the substrate; the agent pattern provides the governance constraint.

42.2 Use Cases

Use Case	Graph Structure	GNN Task
Fraud detection	Applicant → Loans → Transactions	Subgraph anomaly scoring
Risk concentration	Applicant → Loans (multi-hop)	Neighborhood aggregation
Circular guarantees	Loans → Collateral → Loans	Cycle detection + risk
Audit completeness	Resource → Events → Audit records	Path completeness verification

43. Schema Graph as Type System

The schema graph has a remarkable property: because it is static and fully known at system definition time, it can be analyzed before the system runs. This is compile-time validation for institutional processes.

43.1 What Can Be Validated at Definition Time

- State machine well-formedness: no unreachable states, no non-terminal dead ends
- Relationship consistency: if Loan.belongs_to(Applicant), every loan creation must provide applicant_id
- Policy completeness: every state has at least one applicable policy (no ungoverned transitions)
- Invariant feasibility: an invariant referencing edge type "secured_by" is invalid if no resource type declares that relationship
- Role coverage: every transition has at least one role authorized to trigger it (no impossible transitions)
- Agent relevance: agents observing events from a non-existent resource type are flagged

This transforms the schema graph into a type checker for the system definition. Errors are caught before deployment, not at runtime. The DSL becomes a typed language where the schema graph is the type system.

Part VIII: Implementation Roadmap

32. Phase 1: Core Kernel (Current)

~2,600 LOC Python. 53 passing tests.

33. Phase 2: Invariants + Desired State + Storage

- Strong/eventual invariant split + InvariantChecker subsystem
- Desired state primitive + reconciliation loop
- Event log snapshots for efficient replay
- PostgreSQL backends, transaction boundaries, policy indexing
- Resource-level convergence checking in cascade
- Authority level tracking on all transitions

34. Phase 3: Graph Layer + API

- Schema graph: relationship declarations, reachability analysis, deadlock detection
- Instance graph: graph-native state store (neighbors, subgraph, aggregate)
- Temporal graph: causal edges, backward/forward audit traversal
- PostgreSQL + Apache AGE as graph backend
- Graph-expressible invariants (neighborhood aggregation, cycle detection, degree constraints)
- FastAPI REST + WebSocket, OpenTelemetry, SystemTestHarness, CLI

35. Phase 4: GNN + Mature Features

- GNN agent framework: subgraph extraction, feature encoding, inference pipeline
- Cross-resource coordination via graph-aware constraint controllers
- Workflow graphs (parallel paths, joins, branches)
- Policy versioning, temporal replay, multi-tenancy, Kafka
- Neo4j backend option for complex graph query workloads

36. Phase 5: Ecosystem

- Plugin marketplace, visual designer, compliance reports, TypeScript port

Appendices

A. Prior Art

System	What We Take	How We Differ
Kubernetes	Controller-reconciliation, specs, etcd	Richer policies; explicit invariants; graph-native
React	Declarative → reconciled output	Applied to governance, not UI
Temporal	Durable execution	Declarative spec vs imperative workflow
Event Sourcing	Events as ground truth	Dual-layer (event+state+graph) vs pure ES
Kafka	Append-only log, offsets	Embedded in kernel
Neo4j / Property Graphs	Graph-native data model, Cypher	Graph as governance substrate, not just data store
GNNs (MaayaAI)	Graph embedding, anomaly detection	Applied to institutional compliance, not just security

B. CREDIT IQ Mapping

Evaluation Rule	Reconcile Primitive
Rule 2: State Machine	StateMachine + Transitions + Desired State + Schema Graph reachability
Rule 3: Role Complexity	Roles + Permissions + Authority Levels
Rule 4: Policy Density	Policies + Policy Index + Graph-expressible invariants
Rule 5: Observability	AuditLog + EventLog + Temporal Graph (causal traversal)
Rule 6: Temporal	TemporalContext + Policy Versioning + Snapshots
Rule 7: Agent Integration	Agents + GNN agents + Decision Nodes
Rule 8: Combinatorial	States×Roles×Policies×Invariants×Relationships×Time

C. Glossary

Term	Definition
Resource	Domain entity with lifecycle state
Invariant (Strong)	Checked synchronously inside transaction; blocks on failure
Invariant (Eventual)	Enforced asynchronously by controllers; compensates on failure
Desired State	Target state. Controllers push current → desired
Schema Graph	Type-level topology: resource types + relationship declarations
Instance Graph	Live property graph: resource instances + typed edges

Term	Definition
Temporal Graph	Causal event history: events + causality edges
Neighborhood	Set of nodes reachable within depth N from a given node
Graph Predicate	Invariant expressed as a property of a subgraph
Controller	Component enforcing one or more invariants
Agent	AI component: observe → propose (including GNN agents)
Authority Level	Who triggered transition: HUMAN|CONTROLLER|AGENT|SYSTEM
Snapshot	Point-in-time resource state for efficient event replay
Convergence	Cascade steps reduce distance to desired state
Complexity Volume	States×Roles×Policies×Invariants×Relationships×Time

AI commoditizes interfaces. Value lives in system architecture.

Use this specification every time. No exceptions.